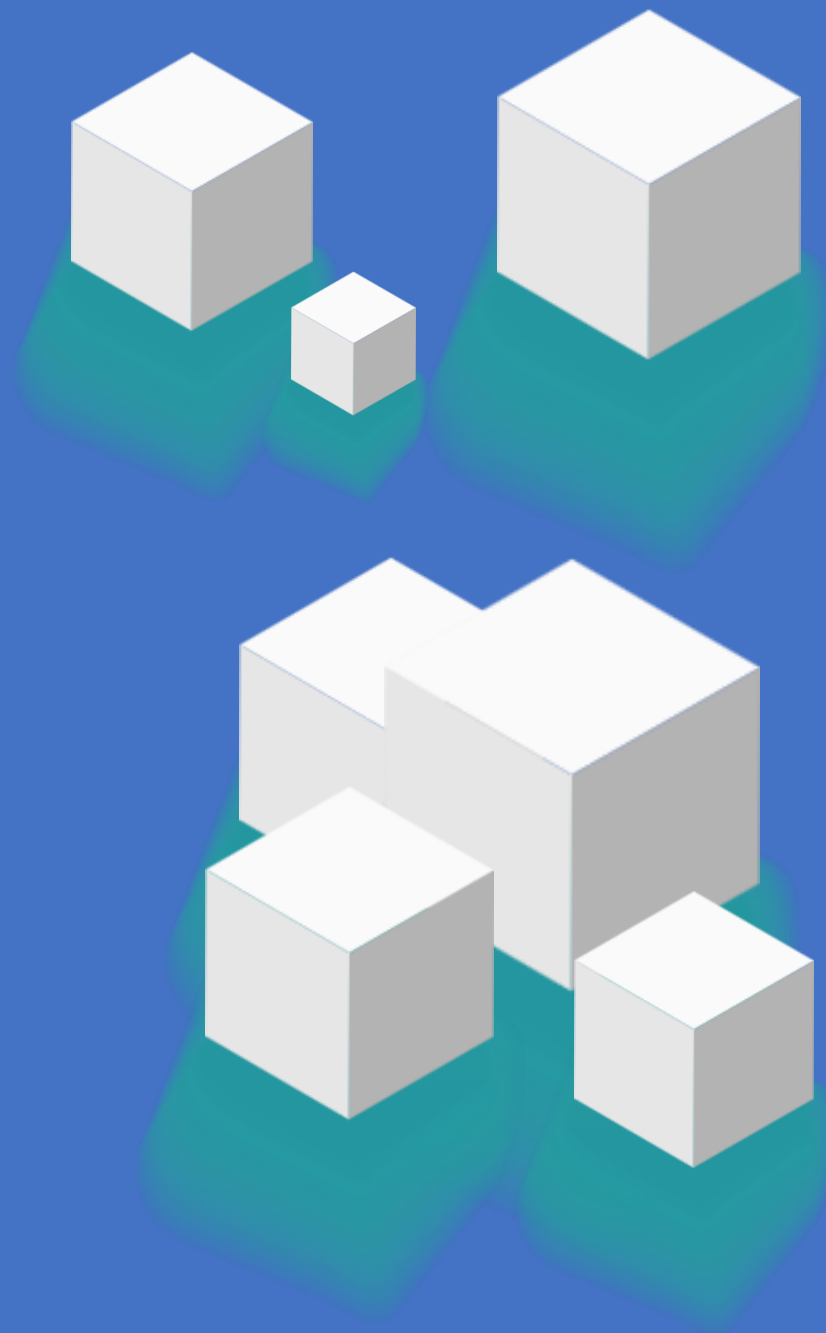
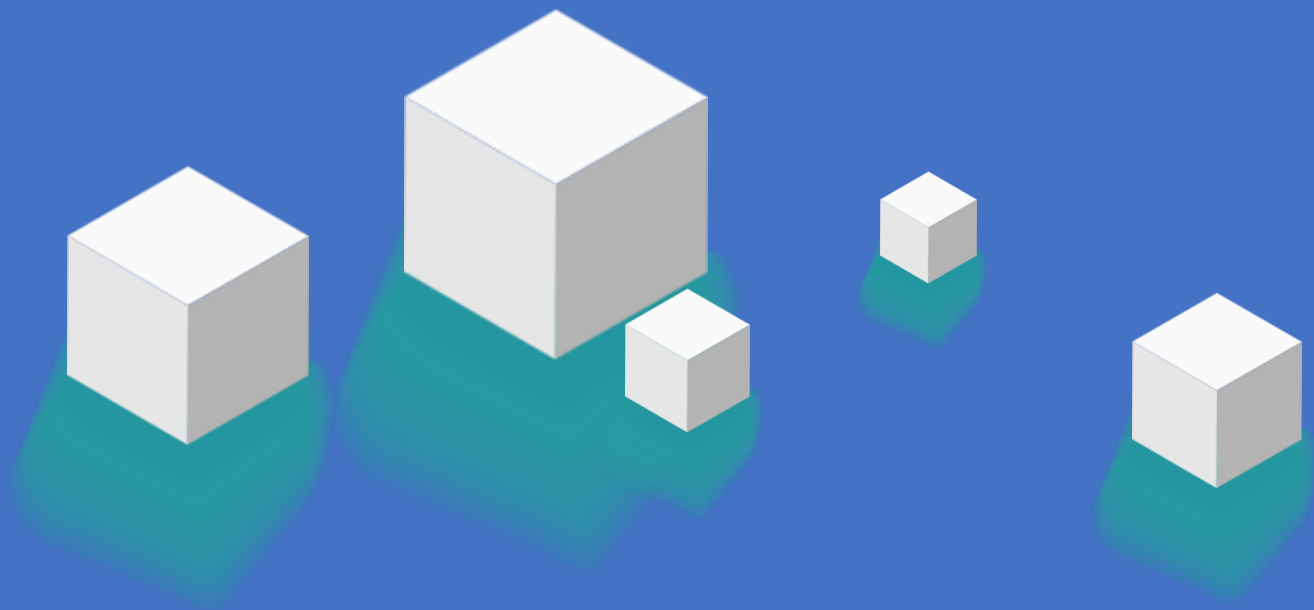


輿情感知与事件驱动



>> 今天的学习目标

舆情感知与事件驱动

- 技能：舆情感知与事件分析

舆情数据获取

金融情感分析

事件驱动系统

宏观恐慌指数监控

Polymarket 预测市场

- Nanobot使用

AgentLoop、ContextBuilder、ToolRegistry、
AgentHook

Skills 渐进式加载机制

自定义工具开发

舆情感知与事件驱动

Thinking: AI投研中的舆情感知与事件驱动都包括哪些？

- 舆情数据获取
- LLM情感分析
- 事件驱动信号
- 宏观恐慌指数
- Polymarket预测市场

=> TO DO: 打造Skill

技能1：輿情数据获取

什么是金融舆情？

Thinking: 什么是金融舆情？

金融舆情 = 市场参与者对某只股票、某个行业或整个市场的集体情绪和看法。它是影响股价短期波动的核心因素之一。

华尔街有句名言: "Buy the rumor, sell the news" (买在传言时, 卖在消息落地时)。

=> 舆情对股价的前瞻性影响 —— 很多时候股价在正式公告之前就已经反应了市场预期。

什么是金融舆情？

Thinking: 机构 vs 散户 是否有信息差？

- 机构投资者有专业的舆情监控系统

24小时扫描新闻、公告、社交媒体 => 第一时间发现资产重组等关键事件。

- 散户通常要等到股吧讨论或者涨停板才知道消息。

目标：用AI缩小信息差 —— 自动抓取新闻、自动分析情感、自动生成交易信号。

舆情数据源

舆情数据的三个层次：

层次	数据源	时效性	可靠性
一手信息	上市公司公告、交易所公告	最快（盘后/盘中）	最高（法定披露）
二手信息	财经新闻（东财、证券时报等）	快（分钟级）	较高（记者采编）
三手信息	股吧、社交媒体、自媒体	最快（秒级）	较低（噪音多）

主要使用一手和二手数据（通过 akshare 获取东方财富新闻和上市公司公告）。

akshare 新闻数据接口

akshare 是一个免费的 Python 金融数据工具库

接口	功能	示例
stock_news_em(symbol)	东方财富个股新闻	stock_news_em("002594")
stock_notice_report(symbol)	上市公司公告	stock_notice_report("002594")
news_cctv(date)	央视新闻（政策面）	news_cctv("20260330")

akshare 新闻数据接口

获取个股新闻

```
import akshare as ak

# 获取比亚迪(002594)的新闻
df = ak.stock_news_em(symbol="002594")
print(f"获取到 {len(df)} 条新闻")
print(df.head())

# 常见字段：关键词、新闻标题、新闻内容、发布时间、文章来源、新闻链接
```

关键词过滤体系

Thinking: 面对海量新闻, 如何快速筛选出对交易有价值的信息?

按事件类型分类关键词:

- **利好类:** 资产重组、回购、业绩预增、股权激励、大额订单、战略合作、增持、扭亏为盈。
- **利空类:** 业绩预减、违规处罚、股东减持、商誉减值、退市风险、诉讼仲裁、亏损。
- **政策类:** 降准降息、产业政策、监管新规、财政刺激、贸易政策。

关键词过滤体系

```
import pandas as pd

def filter_by_keywords(df, keywords, text_columns=None):
    """按关键词过滤新闻"""
    if text_columns is None:
        text_columns = ["新闻标题", "新闻内容", "title", "content"]
        text_columns = [c for c in text_columns if c in df.columns]

    mask = pd.Series([False] * len(df), index=df.index)
    for col in text_columns:
        for kw in keywords:
            mask = mask | df[col].astype(str).str.contains(kw, na=False)

    return df[mask]
```

使用示例

```
bullish_keywords = ["资产重组", "回购", "业绩预增", "增持"]
filtered = filter_by_keywords(news_df, bullish_keywords)
print(f"匹配到 {len(filtered)} 条利好新闻")
```

时间范围筛选

时间范围筛选

```
from datetime import datetime, timedelta

def filter_by_date(df, days=7, date_column="发布时间"):
    """只保留最近N天的新闻"""
    df[date_column] = pd.to_datetime(df[date_column], errors="coerce")
    cutoff = datetime.now() - timedelta(days=days)
    return df[df[date_column] >= cutoff]

# 只看最近7天的新闻
recent_news = filter_by_date(news_df, days=7)
```

去重

新闻来源多样，同一事件可能被多个媒体报道。通过标题去重避免重复分析。

```
def deduplicate(df):
    title_cols = [c for c in df.columns if "标题" in c or "title" in c.lower()]
    if title_cols:
        df = df.drop_duplicates(subset=title_cols, keep="first")
    return df
```

技能2：金融情感分析

金融情感分析

Thinking: 金融情感分析与股票价格之间相关性么？

哥伦比亚大学商学院的金融学教授 Tetlock (2007) 发表在 Journal of Finance 的论文证明：

媒体悲观情绪指数能预测第二天的股市下跌和交易量增加

《Giving Content to Investor Sentiment: The Role of Media in the Stock Market, 2007》，核心发现：

- 使用《华尔街日报》"Abreast of the Market" 专栏的每日内容进行文本分析
- 构建了一个媒体悲观情绪指数 (media pessimism index)
- 发现高媒体悲观情绪预测市场价格的下行压力，随后会回归到基本面
- 极端的悲观或乐观情绪预测高交易量

该论文获得了 2007年 Smith-Breeden 奖（由 Journal of Finance 颁发），这是金融学领域的顶级奖项之一。研究还被《经济学人》杂志专题报道

金融情感分析

摩根大通 在2019年创建 Volfefe Index (推特波动率指数) :

- 追踪特朗普推文对国债波动率的影响, 命中率超过60%
- 命名由来:

名称是 Volatility (波动率) 和 Covfefe 的合成词。Covfefe 是特朗普在2017年5月一条著名推文中的拼写错误 (原意可能是 coverage) , 这个神秘词汇迅速成为网络迷因

Volfe Index

Volfe Index 通过以下方式构建：

- **监测窗口**：追踪推文发布后5分钟内的国债收益率变动
- **滚动概率**：计算每条推文引发市场波动的滚动一月期概率
- **波动率模型**：将指数整合进摩根大通的波动率公允价值模型中

研究发现，市场波动推文通常在美国东部时间中午至下午2点发布

=> 这与美国国债市场流动性最好的时段重合

此外分析表明，特朗普的推文解释了隐含波动率变动中 可测量的部分 (measurable fraction)

Volfe Index

- 高影响关键词:

研究发现包含 China (中国)、billion (十亿)、products (产品)、Democrats (民主党)、great (伟大) 等词汇的推文最可能引发市场波动

- 外汇市场的影响力:

含宏观经济或贸易内容的特朗普推文会导致美元升值、外汇波动率下降, 基于这一信号构建的ETF交易策略在考虑交易成本后夏普比率达到0.69

这个指数本质上是一个**高频事件驱动型工具**, 适用于债券期货、利率衍生品及外汇市场的短期波动率交易。

金融情感分析

SKILL.md 设计

```
---  
name: sentiment-analysis  
description: 监控新闻舆情，分析市场情绪，捕捉事件驱动交易机会  
keywords: 舆情, 情感分析, 新闻监控, 市场情绪, 资产重组, 事件驱动  
---
```

可用脚本

```
| 脚本 | 功能 |  
|-----|-----|  
| scripts/news_fetcher.py | 新闻抓取与过滤 |  
| scripts/sentiment_scorer.py | LLM情感分析 |  
| scripts/event_detector.py | 事件识别与交易信号 |
```

工作流程

```
1. news_fetcher -> 2. sentiment_scorer -> 3. event_detector
```

LLM的优势：

- 理解上下文和金融语义
- 能区分事实与观点
- 可以给出结构化的多维度评分。

金融情感分析

情感分析 Prompt 设计

```
SENTIMENT_PROMPT = """你是一位专业的金融舆情分析师。  
请对以下金融新闻进行情感分析。
```

```
新闻内容：  
{news_text}
```

```
请以严格的JSON格式输出：
```

```
{{  
  "sentiment": "正面/负面/中性",  
  "strength": 1到5的整数（1=极弱, 5=极强） ,  
  "entities": ["相关公司或人物名称列表"],  
  "keywords": ["核心关键词列表"],  
  "summary": "一句话摘要",  
  "market_impact": "对市场可能的影响"  
}}
```

分析要点：

- 区分事实报道与观点评论
- 关注对股价的潜在影响方向
- 识别信息的时效性（已发生/将发生/传闻）
- 注意消息源的可信度"""

金融情感分析

运行流程:

Step 1: 获取新闻 (输入)

先获取比亚迪(002594)最近7天新闻, 生成 JSON 文件

```
python skills/sentiment-analysis/scripts/news_fetcher.py --stock 002594 --days 7
```

=> 输出文件: data/002594_news.json (包含新闻标题、内容、发布时间等字段)

```
[
  {
    "关键词": "002594",
    "新闻标题": "汽车行业资金流出榜：潍柴动力、比亚迪等净流出资金居前",
    "新闻内容": "300863 卡倍亿 3.59 1.43 932.50 603121 华培动力 0.65 3.26 903.65 汽车行业资金流出榜 代码  
简称 今日涨跌幅 (%) 今日换手率 (%) 主力资金流量 (万元) 000338 潍柴动力 -4.86 2.07 -49124.04  
002594",
    "发布时间": "2026-04-07 16:35:00",
    "文章来源": "证券时报网",
    "新闻链接": "http://finance.eastmoney.com/a/202604073696486783.html"
  },
  ...
]
```

data/002594_news.json

金融情感分析

Step 2: 情感分析 (处理)

对新闻文件逐条调用 LLM 分析情感，最后聚合生成整体情绪指数

```
python skills/sentiment-analysis/scripts/sentiment_scorer.py --news_file data/002594_news.json --output_dir output
```

脚本做两件事：逐条分析（每条新闻调用LLM评分）+ 聚合评估（汇总所有分析结果，生成 Fear & Greed Index）。

[开始] 情感分析: data/002594_news.json

[加载] 共 8 条新闻，将分析前 8 条

[分析] (1/8) 300863 卡倍亿 3.59 1.43 932.50 603121 华培动力 0.65 3.26 903.65 汽车行...

[分析] (2/8) 中证智能财讯比亚迪 (002594) 4月1日晚间发布的2026年3月产销快报显示，当月新能源汽车销量为30.02万辆，去年...

[分析] (3/8) 在客户方面，比亚迪 (SZ002594) 成为林泰新材2025年度第一大客户。基于业绩表现，林泰新材计划与投资者共享发展成...

[分析] (4/8) 3月27日晚，比亚迪 (002594.SZ) 发布2025年度报告，多项核心经营指标再创新高：营收8040亿元，净利润326...

[分析] (5/8) 今年1月，前比亚迪 (002594.SZ) 腾势总经理赵长江已宣布加入智界汽车，担任执行董事及执行副总裁。郭锐与拥有丰富汽车...

[分析] (6/8) 其他的瑞风系列、江淮iEV新能源等主力车型，在比亚迪 (002594.SZ)、吉利 (00185.HK)、广汽埃安等头部品牌...

[分析] (7/8) 002532 天山铝业 1 4613.70 -53.95 1.12 有色金属 600066 宇通客车 1 4425.38...

[分析] (8/8) 公司与比亚迪 (002594.SZ)、安瑞光电 (三安光电 (600703.SH) 之全资子公司)、嘉利股份 (874616.NQ...

金融情感分析

Step 3: 逐条分析结果

LLM 对每条新闻输出结构化 JSON, 例如:

```
// 新闻: "王传福回安徽, 同省委书记谈了啥?" -> 正面(强度5)
{
  "sentiment": "正面",
  "strength": 5,
  "entities": ["比亚迪", "王传福", "安徽省委书记"],
  "keywords": ["营收", "净利润", "研发投入", "现金储备"],
  "summary": "比亚迪2025年度报告亮眼, 营收8040亿元, 净利润326亿元",
  "market_impact": "多项核心经营指标创新高, 对股价形成正面支撑"
}
```

```
// 新闻: "汽车行业资金流出榜" -> 负面(强度3)
{
  "sentiment": "负面",
  "strength": 3,
  "entities": ["卡倍亿", "华培动力", "潍柴动力", "比亚迪"],
  "keywords": ["资金流出", "净流出", "汽车行业"],
  "summary": "汽车行业资金流出, 潍柴动力、比亚迪等公司净流出资金居前。",
  "market_impact": "可能对相关公司股价造成短期压力。"
}
```

金融情感分析

情绪指数: Fear & Greed Index

=> 参考 CNN Fear & Greed Index (恐惧与贪婪指数)。0-100分, 0表示极度恐慌, 100表示极度贪婪。

情绪指数并不需要单独运行 -- 它是 sentiment_scorer.py 的最后一步:

逐条分析完成后, 自动将所有分析结果聚合, 调用LLM总结主题、风险和机会。

```
# sentiment_scorer.py 逐条分析完成后, 自动调用该函数聚合

# 基础统计: 正面/负面/中性比例 -> 映射到 0-100
fear_greed_index = int((pos / total) * 100) if total > 0 else 50

# LLM 精细聚合 (在基础统计之上) :
# 调用 LLM 总结 top_themes (主要主题)、risk_alerts (风险点)、
opportunity_hints (交易机会)
```

[skills/sentiment-analysis/scripts/sentiment_scorer.py](https://github.com/justmarktr/skills/sentiment-analysis/scripts/sentiment_scorer.py)

金融情感分析

同一次运行（上面的 Step 2 命令）的聚合输出部分：

[情绪指数] Fear & Greed Index: 50/100

[整体情绪] 中性

[正面/负面/中性] 2/2/1

[主要主题] 汽车行业、比亚迪、江淮汽车

[风险提示] 汽车行业资金流出，部分公司扭亏困难

[机会提示] 比亚迪财报数据亮眼，可能带来投资机会

[摘要] 整体市场情绪趋于中性，正负面信息交织。

=> 两个输出文件：

- output/002594_news_sentiment.json（逐条详细分析）
- output/002594_news_mood.json（聚合情绪指数）

CNN恐惧与贪婪指数

CNN恐惧与贪婪指数：

由 CNN Business 于2012年推出的市场情绪指标，用于量化投资者情绪对股市的影响。

评分范围：0-100分

0-24：极度恐惧 (Extreme Fear)

25-49：恐惧 (Fear)

50：中性 (Neutral)

51-74：贪婪 (Greed)

75-100：极度贪婪 (Extreme Greed)

恐惧会压低股价，贪婪会推高股价。该指数帮助判断市场是否因情绪而偏离合理估值

应用场景

1. 逆向交易策略

巴菲特：在别人恐惧时贪婪，在别人贪婪时恐惧

- 极度恐惧 (≤ 25)：买入机会，市场可能超卖
- 极度贪婪 (≥ 75)：卖出/减仓信号，市场可能过热

经典案例：

2020年3月：COVID-19疫情期间，指数跌至8（历史低位），几天后S&P 500开始史上最快反弹之一

2021年末：指数达到94（极度贪婪），随后成长股开始大幅回调

应用场景

2. 量化交易回测

研究显示，基于该指数的量化策略可获得正收益：

策略规则：每周一根据最新指数值操作

- 指数 ≤ 25 （极度恐惧） $\rightarrow$ 开期货多头
- 指数 ≥ 75 （极度贪婪） $\rightarrow$ 开期货空头
- 其他情况 $\rightarrow$ 空仓

结果：在各国股指期货上测试显示盈利因子为正

应用场景

3. 入场/出场时机确认

结合技术指标使用：

- 买入确认：指数 ≤ 25 + 价格触及强支撑位 + RSI < 30 + MACD金叉
- 卖出确认：指数 ≥ 75 + 价格触及阻力位 + RSI > 70 + MACD死叉

4. 风险管理与仓位控制

- 极端情绪区：降低仓位规模（高波动性）
- 中性区（40-60）：避免交易（信号不明确）
- 止损设置：基于近期波段高低点或ATR设置止损

技能3：事件驱动系统

事件驱动交易

Thinking: 什么是事件驱动交易?

事件驱动交易 (Event-Driven Trading) 是对冲基金和量化机构最常用的策略类别之一。

核心理念: 某些特定事件与股价变动存在统计上的显著相关性, 通过快速识别事件并在市场充分反应之前建仓, 可以获取超额收益。

A股案例:

2024年并购重组潮: 2024年9月证监会发布 并购六条 新规后, A股掀起资产重组浪潮。

据统计, 公告重大资产重组的公司在复牌后首日平均涨幅达到8-15%, 部分公司连续涨停。

=> 如果能在公告发布的第一时间 (甚至之前通过蛛丝马迹预判), 就能获得显著的信息优势

=> 这是事件驱动系统要解决的问题。

事件驱动交易

Thinking: 事件驱动 vs 趋势跟踪 的区别是什么?

趋势策略关注的是：价格在往哪个方向走（技术面）

事件驱动关注的是：发生了什么事会改变价格（消息面）。

=> 两者互补，很多专业机构会同时使用。

事件类型	历史统计影响	交易策略
资产重组	公告后平均涨幅8-15%（A股最强事件类型）	事件确认后关注，但要警惕"利好出尽"
回购增持	中期正面效应（3-6个月）	跟随大股东操作，尤其关注实控人增持
业绩预增	短期正面脉冲（1-5个交易日）	预增幅度>50%更有价值，注意是否已被市场预期
股东减持	短期负面压力	回避或减仓，尤其是实控人减持需高度警惕
违规处罚	视严重程度，可能跌停	立即回避，等尘埃落定

事件识别方案

两层检测策略：

Layer 1: 关键词快速检测（毫秒级）

基于预定义的关键词库，高效过滤，适合大批量新闻。

Layer 2: LLM精细识别（秒级）

对关键词未命中的新闻做二次检测，理解复杂语义和隐含事件。

事件识别方案

```
EVENT_KEYWORDS = {
    "利好": {
        "资产重组": ["资产重组", "重大资产", "借壳上市", "资产注入"],
        "回购增持": ["回购", "增持", "股份回购", "大股东增持"],
        "业绩预增": ["业绩预增", "业绩大增", "净利润增长", "扭亏为盈"],
    },
    "利空": {
        "股东减持": ["减持", "股东减持", "高管减持", "清仓"],
        "业绩预减": ["业绩预减", "业绩下滑", "亏损", "营收下降"],
        "违规处罚": ["违规", "处罚", "立案调查", "行政处罚"],
    },
}
```

```
def keyword_based_detection(news_text):
    """基于关键词的快速事件检测"""
    detected = []
    for event_type, categories in EVENT_KEYWORDS.items():
        for category, keywords in categories.items():
            matched = [kw for kw in keywords if kw in news_text]
            if matched:
                detected.append({
                    "event_type": event_type,
                    "event_category": category,
                    "matched_keywords": matched,
                })
    return detected
```

事件识别方案

事件 -> 交易信号映射

```
SIGNAL_MAP = {
    "利好": {
        "资产重组": {"signal": "强烈关注", "reason": "资产重组可能带来基本面质变"},
        "回购增持": {"signal": "看多", "reason": "大股东用真金白银表达信心"},
        "业绩预增": {"signal": "看多", "reason": "业绩超预期增长"},
    },
    "利空": {
        "股东减持": {"signal": "谨慎", "reason": "内部人士减持可能释放负面信号"},
        "业绩预减": {"signal": "看空", "reason": "基本面恶化"},
        "退市风险": {"signal": "强烈回避", "reason": "退市风险极高"},
    },
}

def generate_signal(event_type, event_category):
    """根据事件生成交易信号"""
    type_signals = SIGNAL_MAP.get(event_type, {})
    signal = type_signals.get(event_category, {"signal": "观望", "reason": "信息不足"})
    return signal
```

[skills/sentiment-analysis/scripts/event_detector.py](https://github.com/quantopian/skills/sentiment-analysis/scripts/event_detector.py)

事件驱动交易

运行结果（实际输出）

搜索"资产重组,回购"关键词，最近3天：

[事件] (利好) 资产重组 -> 强烈关注 | 预计本次交易可能构成重大资产重组。亿纬锂能（300014） ...

[事件] (利好) 资产重组 -> 强烈关注 | 长源东谷表示，本次交易构成关联交易，预计构成重大资产重组...

[事件] (利好) 资产重组 -> 强烈关注 | 安诺其：筹划重大资产重组事项 8日起停牌...

[事件] (利好) 资产重组 -> 强烈关注 | 证券日报e公司讯 4月7日，苏豪时代发布停牌公告...

[事件] (利好) 资产重组 -> 强烈关注 | 证券日报e公司讯 4月6日，广博文化在互动平台回复...

[检测结果] 共检测到 8 个事件

利好: 8 个 | 利空: 0 个 | 政策: 0 个

技能4：宏观恐慌指数监控

宏观恐慌指数监控

Thinking: 为什么需要宏观维度的恐慌指数监控?

前面的微观舆情（个股新闻+事件检测）解决了这只股票怎么样的问题，
但投资还需要回答：现在市场整体环境如何。

=> 一只好股票在系统性风险面前也可能大跌

微观舆情好比看天气预报决定穿什么衣服，宏观指标好比看季节判断应不应该出门。
再好的衣服，台风天也别出门。

VIX：华尔街的恐慌温度计

VIX：

全称 CBOE Volatility Index（芝加哥期权交易所波动率指数），由芝加哥期权交易所于1993年推出。它衡量的是未来30天标普500指数期权的隐含波动率。

VIX的本质：市场参与者为了对冲风险而购买保险（看跌期权）的价格。当所有人都急着买保险时，保险价格（VIX）就会飙升 -- 这正说明市场非常恐慌。

历史极端值参考：

- 2008年金融危机：VIX 最高 89.53（雷曼兄弟倒闭后）
- 2020年3月 COVID-19：VIX 最高 82.69
- 2022年俄乌冲突：VIX 最高约 36
- 正常牛市环境：VIX 通常在 12-18

VIX 与标普500大约呈 -0.8 的负相关。

VIX 飙升 = 股市暴跌，反之亦然。

但 VIX 有均值回归特性，即极端恐慌后往往是买入良机。

OVX 和 GVZ

OVX 和 GVZ -- VIX 的兄弟指标

- OVX (原油波动率指数) :

衡量原油ETF期权的隐含波动率。

原油是全球经济的血液，OVX飙升往往意味着地缘政治紧张（中东冲突、贸易制裁等）或供应链危机。

- GVZ (黄金波动率指数) :

衡量黄金ETF期权的隐含波动率。

黄金是传统避险资产，GVZ升高反映市场对不确定性的担忧加剧，资金正在涌向避风港。

美国10年期国债收益率

美国10年期国债收益率 -- 全球资产定价之锚

Thinking: 为什么10年期国债这么重要?

因为它是DCF（现金流折现模型）中无风险利率的基准。几乎所有资产的定价都离不开这个数字

DCF估值公式：股票价值 = 未来现金流 / (1 + 无风险利率 + 风险溢价)ⁿ

逻辑链条：

10年期国债收益率上升 -> 分母变大 -> 同样的未来现金流折算到今天的价值变低 -> 股票估值下降

这就是为什么加息周期中，高估值的成长股（如科技股）跌幅最大 -- 因为它们的价值主要来自遥远未来的现金流，对利率最敏感。

而价值股（如银行、能源）受影响较小，因为它们的现金流主要在近期。

利率上升利价值股，利率下降利成长股。4.3%是近年来的重要分水岭。

美国10年期国债收益率

美国10年期国债收益率：

被称为全球资产定价之锚，直接影响成长股和价值股的估值对比。

区间	含义	资产配置
> 4.4%	利率偏高	看好价值股和防御板块
4.3-4.4%	分水岭	密切关注方向选择
< 4.3%	宽松预期	资金回流成长股

多维度恐慌指标体系

构建多维度恐慌指标体系：

指标	含义	关注重点	获取方式
VIX	美股恐慌指数（标普500隐含波动率）	市场整体恐慌程度	akshare / Yahoo Finance
OVX	原油ETF波动率指数	地缘政治和能源供应风险	Yahoo Finance
GVZ	黄金ETF波动率指数	避险情绪强度	Yahoo Finance
US10Y	美国10年期国债收益率	全球资产定价基准，成长vs价值风格切换	akshare / Yahoo Finance

多维度恐慌指标体系

VIX 恐慌指数阈值

区间	含义	操作建议
< 15	极度平静（警惕自满）	市场可能过于乐观
15-20	正常	按策略正常操作
20-25	焦虑	适当控制仓位
25-35	恐慌	关注超跌反弹机会
> 35	极度恐慌	历史上往往是中长期买入良机

风险传导逻辑：

- 当 OVX飙升但VIX滞后 → 风险仍集中在能源端，尚未传导至全球宏观信用风险
- 当 OVX与VIX同步共振向上 → 地缘风险已触发流动性危机或全球经济衰退预期，需立即风控

多维度恐慌指标体系

综合恐慌/贪婪评分：

将多个指标汇总为 0-100 的综合评分（类似 CNN Fear & Greed Index），方便快速判断市场情绪。

```
score = 50 # 基准分(中性)
# VIX 维度
if vix < 15: score += 30 # 极度贪婪
elif vix < 20: score += 15 # 偏贪婪
elif vix < 25: score += 0 # 中性
elif vix < 35: score -= 15 # 恐慌
else: score -= 30 # 极度恐慌

# 10年期国债维度
if us10y < 3.8: score += 10 # 宽松利好
elif us10y > 4.8: score -= 10 # 紧缩利空
# 最终映射到 0-100
score = max(0, min(100, score))
```

[skills/sentiment-analysis/scripts/market_fear_index.py](https://github.com/quantopian/skills/sentiment-analysis/scripts/market_fear_index.py)

多维度恐慌指标体系

运行结果: `python skills/sentiment-analysis/scripts/market_fear_index.py --output_dir output`

[VIX] 当前值: 25.63, 风险等级: 恐慌 (来源: Yahoo Finance)

[10Y国债] 收益率: 4.337%, 策略: 利率处于分水岭附近, 密切关注方向选择
(来源: Yahoo Finance)

[OVX] 当前值: 96.14 (来源: Yahoo Finance)

[GVZ] 当前值: 37.04 (来源: Yahoo Finance)

综合恐慌/贪婪指数: 25/100

整体情绪: 极度恐慌

建议操作: 市场极度恐慌, 历史上往往是中长期买入良机

[VIX] 25.63 -> VIX偏高, 市场存在恐慌情绪

[US10Y] 4.337 -> 利率处于分水岭, 密切关注

[OVX] 96.14 -> 原油波动率高, 地缘政治风险可能升级

[风险传导] OVX与VIX同步共振向上

解读: 地缘风险已触发流动性危机或全球经济衰退预期

操作: 需立即风控, 增配黄金和现金类资产

[skills/sentiment-analysis/scripts/market_fear_index.py](#)

Polymarket 预测市场

Polymarket 预测市场

Thinking: 预测市场的经济学原理是什么?

预测市场 (Prediction Market) 的核心思想源自有效市场假说:

当大量参与者用真金白银押注某个事件的结果时, 市场价格会自动汇聚为最准确的概率估计。

五角大楼披萨指数:

冷战时期, 情报分析师发现了一个有趣的现象: 每当国际局势紧张时, 五角大楼 (美国国防部) 附近的披萨外卖订单量就会飙升 -- 因为军方高层加班开会, 只能叫外卖。

=> 于是 五角大楼附近的深夜披萨订单 成了判断是否要开战的民间指标。

真正有价值的信号, 往往藏在意想不到的数据中。预测市场就是把这种用脚投票的行为标准化

Polymarket 预测市场

Thinking: 预测市场 vs 传统预测的区别是什么？

经济学家 Robin Hanson 在2003年提出，预测市场是信息聚合机器

它比专家预测更准确，因为它让所有参与者有动力说真话（说错了要亏钱）。

2024年美国大选，Polymarket的预测准确度远超传统民调，引发华尔街高度关注。

Polymarket：全球最大的去中心化预测市场，基于 Polygon 区块链运行。

用户用真金白银（USDC）押注事件发生的概率。2026年被洲际交易所（纽交所母公司）投资20亿美元，估值90亿美元。

Polymarket 预测市场

Thinking: 为什么预测市场对量化交易有价值?

因为传统的新闻分析是回顾性的（已发生的事），而预测市场是前瞻性的（即将发生的事的概率定价）。

当一个事件的概率从 20% 突然跳到 60%，往往意味着有聪明钱基于内部信息在押注。

维度	Polymarket	传统民调/分析师
资金门槛	无上限（曾出现单笔百万美元押注）	不涉及真金白银
透明度	全链上公开可查	不透明
更新频率	实时（每分钟更新）	天/周级
API开放度	完全开放，无需认证	多数需申请

Gamma API 数据获取

```
import httpx
import json

GAMMA_API = "https://gamma-api.polymarket.com"

# 搜索地缘政治相关预测市场
resp = httpx.get(f"{GAMMA_API}/events", params={
    "active": "true",
    "closed": "false",
    "search": "Iran", # 搜索关键词
    "limit": 20,
})
events = resp.json()
```

```
# 解析每个事件的市场数据
for event in events:
    for market in event.get("markets", []):
        prices = json.loads(market.get("outcomePrices", "[0,0]"))
        yes_pct = float(prices[0]) * 100
        volume = float(market.get("volume", 0))
        print(f"{market['question']}")
        print(f" Yes: {yes_pct:.1f}% | 交易量: ${volume:,.0f}")
```

事件概率 → 资产配置

事件概率 → 资产配置

Polymarket 信号	资产操作	逻辑
开战概率 > 60%	做多黄金/原油，做空科技股	避险资产上涨
停火概率快速上升	平仓原油多头，买入被制裁国有资产	风险情绪修复
概率长期僵持(40%-60%)	做多波动率（买入VIX看涨期权）	不确定性溢价
关税概率 > 60%	回避出口型企业，关注内需板块	关税冲击出口

聪明钱信号检测：当一个市场突然出现大额集中押注（交易量远超平均值3倍以上），且概率高度倾斜（>85%），通常意味着有内幕信息在流通。

2026年2月27日，150+个新账户集中押注次日开战，总金额约85.5万美元。2月28日美军空袭伊朗，这些账户获利超120万美元。

事件概率 → 资产配置

运行结果: `python skills/sentiment-analysis/scripts/polymarket_monitor.py --keyword "tariff" --min_volume 100000 --output_dir output`

[Polymarket] 关键词 'tariff' 找到 20 个事件, 解析出 58 个活跃市场

[聪明钱信号] 检测到 41 个信号 (交易量远超平均值或概率高度倾斜的市场)

[交易量 Top 5]

1. MicroStrategy sells any Bitcoin in 2025?

Yes: 0.0% | 交易量: \$18.0M | 极强否定

2. GTA VI released before June 2026?

Yes: 1.6% | 交易量: \$13.2M | 极强否定

3. Will Jesus Christ return before GTA VI?

Yes: 48.5% | 交易量: \$10.9M | 不确定 (波动率机会)

4. Will China invades Taiwan before GTA VI?

Yes: 52.5% | 交易量: \$1.8M | 不确定 (波动率机会)

5. Starmer out by December 31, 2026?

Yes: 55.5% | 交易量: \$890.1K | 不确定 (波动率机会)

提示: Polymarket API 按事件级别搜索, 返回事件内所有子市场。

实际使用时可结合关键词+交易量阈值进一步筛选。

多维决策：多Skill联动

多维决策：多Skill联动

从单一指标到多维决策：

专业投资机构做一个投资决策，通常需要经过 宏观 -> 中观 -> 微观 三个层次的分析

层次	关注问题	对应工具
宏观	全球经济环境如何？有没有系统性风险？	market_fear_index.py (VIX/US10Y)
中观（前瞻）	未来会发生什么？地缘政治走势？	polymarket_monitor.py
微观	这只股票怎么样？有什么事件？	news_fetcher.py + sentiment_scorer.py + event_detector.py

Thinking：单看任何一个指标会有误判么？

比如个股利好消息出来，但恰逢VIX飙升（系统性恐慌），大概率也会被拖累。

=> 多 Skill 联动，形成完整的决策链。

场景1：宏观风险 + 个股舆情联动

联动流程：多个Skill协同工作，形成 微观+宏观+前瞻 的完整事件驱动体系。

场景1：宏观风险 + 个股舆情联动

先看宏观环境是否友好，再看个股消息面。

你: 现在适合买入A股吗？帮我做个全面评估

AI投研:

1. [market_fear_index.py] 获取全球恐慌指标...

VIX: 18.5 (正常) | 10Y国债: 4.35% (分水岭)

综合评分: 65/100 (偏贪婪)

2. [polymarket_monitor.py] 检查地缘政治风险...

中美关税概率: 72% (偏高, 需警惕)

3. [news_fetcher.py] 获取A股热点新闻...

4. [sentiment_scorer.py] 情感分析...

整体情绪: 乐观 (Fear & Greed: 68)

结论：全球宏观环境中性偏好，但需关注中美关税风险。

A股情绪偏乐观，建议适度参与但控制仓位在60%以内。

场景2：事件驱动 + 深度研报联动

场景2：事件驱动 + 深度研报联动

sentiment-analysis 发现事件 → read-pdf 读取财报 → write-report 生成五步法报告。

你：最近有没有资产重组的消息？帮我深度分析

AI投研：

1. [news_fetcher.py] 搜索"资产重组"关键词...
检测到 [XX公司] 资产重组方案
信号：强烈关注 | 情绪：正面（强度4/5）
2. [read-pdf] 正在读取XX公司最新年报...
3. [write-report] 按五步法分析...
 - 信息差 / 逻辑差 / 预期差 / 催化剂 / 结论
4. 研报已生成：reports/XX公司_事件驱动研报.md

Agent 层面的联动由 nanobot 框架的 Skills 渐进式加载机制自动处理。

Agent 读取 sentiment-analysis/SKILL.md 后知道可以联动 read-pdf 和 write-report

LLM 根据用户需求自行决定调用哪些脚本、按什么顺序执行。

多维决策：多Skill联动

Thinking：多Skill联动带来了什么价值？

- 数据全链路：宏观风险 + 预测市场 + 个股舆情 + 历史财报 + 深度分析。
- 决策完整性：全球环境 + 事件 + 基本面 + 投资建议。
- 前瞻性：Polymarket 提供事件的概率定价，而不仅是已发生的新闻。
- 效率提升：原有分析师流程的AI化，AI投研几分钟完成。

CASE：场景实战

场景1：我持有比亚迪，最近消息面怎么样？

场景1：我持有比亚迪，最近消息面怎么样？

投资者的困惑："我持有比亚迪(002594)的股票，最近好像有不少新闻，但上班太忙没时间一条条看。能不能帮我快速了解一下最近的舆情？是偏正面还是负面？ "

=>解决思路：获取最近7天的新闻 -> LLM逐条分析情感 -> 聚合生成整体情绪评分。

Step 1：获取新闻

```
python skills/sentiment-analysis/scripts/news_fetcher.py --stock 002594 --days 7
```

[获取] 个股新闻: 002594, 共 10 条

[时间] 最近 7 天, 筛选 9/10 条

[去重] 9 -> 9 条

[结果] 输出: data/002594_news.json, 列: 关键词|新闻标题|新闻内容|

发布时间|文章来源|新闻链接

场景1：我持有比亚迪，最近消息面怎么样？

Step 2: 情感分析 + 情绪指数

```
python skills/sentiment-analysis/scripts/sentiment_scorer.py --news_file data/002594_news.json --output_dir output
```

[情绪指数] Fear & Greed Index: 50/100

[整体情绪] 中性

[正面/负面/中性] 2/2/1

[主要主题] 汽车行业、比亚迪、江淮汽车

[风险提示] 汽车行业资金流出，部分公司扭亏困难

[机会提示] 比亚迪财报数据亮眼，可能带来投资机会

[摘要] 整体市场情绪趋于中性，正负面信息交织。

=> 结论：比亚迪近期舆情中性，正面（财报亮眼）和负面（行业资金流出）交织。不用恐慌也不用追高，维持持仓观望即可。

场景2：最近A股有没有重大事件值得关注？

场景2：最近A股有没有重大事件值得关注？

投资者的需求：我是事件驱动投资者，主要关注资产重组和回购类事件。最近A股有没有这方面的机会？

=> 解决思路：按事件关键词搜索新闻 -> 事件检测器自动识别并生成交易信号。

Step 1：搜索事件关键词

```
python skills/sentiment-analysis/scripts/news_fetcher.py --keywords 资产重组,回购 --days 3
```

[任务] 搜索关键词: ['资产重组', '回购']

[时间] 最近 3 天，筛选 8/10 条

[结果] 输出: data/资产重组_回购_news.json

场景2：最近A股有没有重大事件值得关注？

Step 2: 事件检测 + 交易信号

```
python skills/sentiment-analysis/scripts/event_detector.py --news_file data/资产重组_回购_news.json --output_dir output
```

[检测结果] 共检测到 8 个事件

利好: 8 个 | 利空: 0 个 | 政策: 0 个

[交易信号]

[利好] 资产重组 -> 强烈关注 (资产重组可能带来基本面质变)

-> 300067 拟重大资产重组！亿纬锂能50GWh储能电池基地

[利好] 资产重组 -> 强烈关注 (资产重组可能带来基本面质变)

-> 603950 长源东谷宣布重大资产重组！明天复牌！

[利好] 资产重组 -> 强烈关注 (资产重组可能带来基本面质变)

-> 安诺其：筹划重大资产重组事项 8日起停牌

[利好] 资产重组 -> 强烈关注 (资产重组可能带来基本面质变)

-> 苏豪时代发布停牌公告，筹划资产收购

...

=> 结论：最近3天检测到8个资产重组事件，全部利好。值得进一步研究的标的包括亿纬锂能(300067)、长源东谷(603950)等。下一步可以联动 read-pdf 读取财报、write-report 生成深度研报。

场景3：现在全球市场情绪如何？适合加仓吗？

场景3：现在全球市场情绪如何？适合加仓吗？

投资者的疑问：最近新闻说美股在跌，中东局势也不太平。现在全球市场恐慌程度怎么样？我的A股仓位要不要减一减？

=> 解决思路：一键获取 VIX、OVX、GVZ、美国10年期国债收益率，自动生成综合恐慌/贪婪评分。

```
python skills/sentiment-analysis/scripts/market_fear_index.py --output_dir output
```

综合恐慌/贪婪指数: 25/100

整体情绪: 极度恐慌

建议操作: 市场极度恐慌，历史上往往是中长期买入良机

[VIX] 25.63 -> VIX偏高，市场存在恐慌情绪

[US10Y] 4.337 -> 利率处于分水岭，密切关注

[OVX] 96.14 -> 原油波动率高，地缘政治风险可能升级

[风险传导] OVX与VIX同步共振向上

解读: 地缘风险已触发流动性危机或全球经济衰退预期

=> 结论：全球市场正处于极度恐慌状态（25/100），VIX和OVX同步飙升说明不只是能源问题，已经传导到整体市场。短期内建议控制仓位、增配防御性资产。但从历史规律看，极度恐慌往往是中长期的买入窗口。

场景4：最近的关税政策会影响我的投资吗？

场景4：最近的关税政策会影响我的投资吗？

投资者的担忧：最近看到很多关于中美关税的新闻，聪明钱怎么看这件事？市场定价的概率有多高？我需要调整出口股的持仓吗？

=> 解决思路：查询 Polymarket 预测市场，看市场参与者用真金白银定价的概率。

```
python skills/sentiment-analysis/scripts/polymarket_monitor.py --keyword "tariff" --min_volume 100000 --output_dir output
```

[Polymarket] 关键词 'tariff' 找到 20 个事件，58 个活跃市场

[聪明钱信号] 检测到 41 个信号

[结果] 保存至 output/polymarket_20260407.json

=> 结论：关税相关话题在 Polymarket 非常活跃（58个市场），且聪明钱信号密集。

需要打开 JSON 文件查看具体概率。如果关税落地概率 > 60%，建议回避出口型企业，关注内需板块。

场景5：全链路决策（微观+宏观+前瞻）

场景5：全链路决策（微观+宏观+前瞻）

用户问题："我手里有一笔资金，想做一次完整的投资决策。现在市场环境怎么样？有没有值得关注的事件？哪只股票可以深入研究？"

解决思路：按"宏观 -> 前瞻 -> 微观 -> 深度"四步走。

完整流程：

1. 宏观恐慌评估 (market_fear_index.py)
2. 预测市场扫描 (polymarket_monitor.py)
3. 个股事件发现 (news_fetcher.py + event_detector.py)
4. 联动 read-pdf 读取财报
5. 联动 write-report 生成研报



Nanobot

Nanobot

Nanobot:

<https://github.com/HKUDS/nanobot>

由香港大学数据科学实验室 (HKUDS) 开发的一个超轻量级个人 AI 助手，是 OpenClaw 的精简版实现

- **极致轻量**：核心代码仅约 4,000 行，比 OpenClaw 小 99%，资源占用极低，启动飞快
- **研究友好**：代码简洁易读，方便修改和扩展
- **开箱即用**：支持一键部署，通过 pip、uv 或源码即可安装
- **多平台支持**：内置 WhatsApp、Telegram、Discord、Matrix、Slack、飞书、QQ 等聊天渠道



The screenshot shows the GitHub repository page for nanobot. At the top, there are navigation links: README, Contributing, MIT license, and Security. The repository name 'nanobot' is displayed with an orange cat logo. Below the name, it says 'nanobot: Ultra-Lightweight Personal AI Agent'. A bar of badges shows various statistics: pypi v0.1.4.post6, downloads 145k, python >=3.11, license MIT, Feishu Group, WeChat Group, Discord, and Community. The description states: 'nanobot is an ultra-lightweight personal AI assistant inspired by OpenClaw.' It highlights that it 'Delivers core agent functionality with 99% fewer lines of code.' and provides a real-time line count command: 'Real-time line count: run bash core_agent_lines.sh to verify anytime.' The 'News' section lists recent updates from 2026-04-02 to 2026-03-27, including improvements in long-running tasks, GitHub Copilot auth, WeChat multimodal alignment, OpenAI-compatible API, WeChat voice, provider docs, and the release of v0.1.4.post6 with architecture decoupling and security fixes.

nanobot: Ultra-Lightweight Personal AI Agent

nanobot is an ultra-lightweight personal AI assistant inspired by [OpenClaw](#).

Delivers core agent functionality with 99% fewer lines of code.

Real-time line count: run `bash core_agent_lines.sh` to verify anytime.

News

- 2026-04-02 📦 Long-running tasks run more reliably — core runtime hardening.
- 2026-04-01 🛠️ GitHub Copilot auth restored; stricter workspace paths; OpenRouter Claude caching fix.
- 2026-03-31 🗨️ WeChat multimodal alignment, Discord/Matrix polish, Python SDK facade, MCP and tool fixes.
- 2026-03-30 🌱 OpenAI-compatible API tightened; composable agent lifecycle hooks.
- 2026-03-29 🗨️ WeChat voice, typing, QR/media resilience; fixed-session OpenAI-compatible API.
- 2026-03-28 📄 Provider docs refresh; skill template wording fix.
- 2026-03-27 🚀 Released v0.1.4.post6 — architecture decoupling, litellm removal, end-to-end streaming, WeChat channel, and a security fix. Please see [release notes](#) for details.

Nanobot

主要功能：

- **工具调用：**支持 MCP 工具服务器，可执行代码、搜索网络、操作文件等
- **定时任务：**内置 Cron 支持，可设置周期性提醒和自动化 workflow
- **记忆系统：**支持持久化记忆，长期保存重要上下文
- **多模型兼容：**支持 OpenRouter、本地 vLLM 等任意 OpenAI 兼容接口
- **多实例运行：**可同时运行多个针对不同平台或不同用途的机器人实例

Nanobot VS DeepAgents

与DeepAgents相比:

- **配置优先:** 一个 config.json 即可定义完整 Agent (模型、工具、提供商)
- **Skills 渐进式加载:** SKILL.md 文件自动发现, 按需注入 prompt
- **原生多模型支持:** 内置 DashScope(Qwen)、DeepSeek、OpenAI、Anthropic 等 20+ 提供商
- **SDK + CLI 双入口:** 既可以 nanobot 命令行运行, 也可以 Python SDK 编程调用

Nanobot = config.json(配置) + AGENTS.md(身份) + skills/(技能) + Tool 类(工具) -- 组合即 Agent。

Nanobot VS DeepAgents

维度	Nanobot	DeepAgents
框架体量	轻量，纯 Python	重型，依赖 LangGraph + LangChain
配置方式	config.json 声明式	Python 代码构建
Skills 机制	原生渐进式加载（SKILL.md 自动发现）	SkillsMiddleware 中间件注入
工具定义	继承 Tool 基类，显式注册	@tool 装饰器 + 中间件管理
Qwen 支持	原生支持（providers.dashscope）	需手动替换为 ChatTongyi
子 Agent	spawn 工具 + SubagentManager	task() 工具 + SubAgentMiddleware
上下文管理	ContextBuilder + MemoryConsolidator	SummarizationMiddleware
入口方式	CLI (nanobot) / Python SDK	纯 Python 脚本

Nanobot VS DeepAgents

与DeepAgents相比:

- **配置优先:** 一个 config.json 即可定义完整 Agent (模型、工具、提供商)
- **Skills 渐进式加载:** SKILL.md 文件自动发现, 按需注入 prompt
- **原生多模型支持:** 内置 DashScope(Qwen)、DeepSeek、OpenAI、Anthropic 等 20+ 提供商
- **SDK + CLI 双入口:** 既可以 nanobot 命令行运行, 也可以 Python SDK 编程调用

Nanobot = config.json(配置) + AGENTS.md(身份) + skills/(技能) + Tool 类(工具) -- 组合即 Agent。

Nanobot架构

整体架构：



Nanobot架构: AgentLoop

AgentLoop -- 核心运行时

所有逻辑的入口。接收消息 -> 构建上下文 -> 调用 LLM -> 执行工具 -> 返回响应。

```
class AgentLoop:
    def __init__(self, bus, provider, workspace, model, ...):
        self.context = ContextBuilder(workspace) # System Prompt 组装
        self.tools = ToolRegistry()             # 工具注册表
        self.runner = AgentRunner(provider)      # LLM+工具循环
        self.subagents = SubagentManager(...)   # 子 Agent 管理
        self._register_default_tools()          # 注册内置工具
```

Nanobot架构: ContextBuilder

ContextBuilder -- System Prompt 组装

从多个来源组装完整的 System Prompt:

```
class ContextBuilder:
    def build_system_prompt(self):
        parts = [self._get_identity()]          # 1. 内置身份描述
        parts.append(self._load_bootstrap_files()) # 2. AGENTS.md / SOUL.md
        parts.append(self.memory.get_memory_context()) # 3. memory/MEMORY.md
        parts.append(always_skills)             # 4. always=true 的技能
        parts.append(self.skills.build_skills_summary()) # 5. 技能 XML 摘要
        return "\n\n".join(parts)
```

渐进式加载的关键: ContextBuilder 只将技能的名称和描述 (XML 摘要) 注入 prompt, Agent 需要时通过 read_file 工具读取完整 SKILL.md 内容。

=> 避免了将所有技能全文塞入 prompt 导致 token 浪费。

Nanobot的原生渐进式设计

Nanobot 原生设计了渐进式加载，而且设计得很优雅。是一个三级加载系统：

第 1 级 -- 元数据 (永远在 context 中, ~100 words/skill)

SkillsLoader.build_skills_summary() 把所有 skill 生成 XML 摘要注入 system prompt:

```
lines = ["<skills>"]
for s in all_skills:
    name = escape_xml(s["name"])
    path = s["path"]
    desc = escape_xml(self._get_skill_description(s["name"]))
    skill_meta = self._get_skill_meta(s["name"])
    available = self._check_requirements(skill_meta)

    lines.append(f" <skill available='{str(available).lower()}'>")
    lines.append(f" <name>{name}</name>")
    lines.append(f" <description>{desc}</description>")
```

```
lines.append(f" <location>{path}</location>")
// ...
lines.append(" </skill>")
lines.append("</skills>")
```

Nanobot的原生渐进式设计

第 2 级 -- always 技能 (全文自动注入)

frontmatter 中标记 `always: true` 的技能，全文自动塞进 system prompt:

```
always_skills = self.skills.get_always_skills()
if always_skills:
    always_content = self.skills.load_skills_for_context(always_skills)
    if always_content:
        parts.append(f"# Active Skills\n\n{always_content}")
```

`always = true`，代表这个技能的完整内容永远存在于 system prompt 中，不需要模型主动去 read_file

目前内置只有 memory 技能标记了 `always: true`:

```
---
name: memory
description: Two-layer memory system with grep-based recall.
always: true
---
```

Nanobot的原生渐进式设计

第 3 级 -- 按需加载 (模型自主 read_file)

system prompt 中注入提示, 告诉模型需要时去 read_file:

```
skills_summary = self.skills.build_skills_summary()
    if skills_summary:
        parts.append(f""""# Skills
```

The following skills extend your capabilities. To use a skill, read its SKILL.md file using the read_file tool.
Skills with available="false" need dependencies installed first - you can try installing them with apt/brew.

Nanobot的原生渐进式设计

框架的 skill-creator 明确写了这个设计原则

```
### Progressive Disclosure Design Principle
```

Skills use a three-level loading system to manage context efficiently:

1. **Metadata (name + description)** - Always in context (~100 words)
2. **SKILL.md body** - When skill triggers (<5k words)
3. **Bundled resources** - As needed by the agent (Unlimited)



渐进式披露设计原则

技能采用三级加载系统来高效管理上下文：

1. 元数据（名称+描述） — 始终处于上下文中（约100词）
2. SKILL.md 主体 — 技能触发时加载（少于5000词）
3. 捆绑资源 — 按Agent需要加载（无限制）

Nanobot的原生渐进式设计

完整 system prompt 组装顺序

1. # nanobot (身份 + workspace路径 + 平台策略 + 行为准则)

2. ## AGENTS.md (用户定义的角色/指令)

3. # Memory (MEMORY.md 长期记忆)

4. # Active Skills (always=true 的技能全文) <-- 第2级

5. # Skills <-- 第1级(XML摘要) + 第3级(提示read_file)

<skills>

<skill available="true">

<name>...</name>

<description>...</description>

<location>绝对路径/SKILL.md</location>

</skill>

</skills>

框架简洁度对比

维度	nanobot	deepagents
代码量	skills.py 229行, context.py 220行	skills.py 834行 + graph.py + middleware
skill 格式	YAML frontmatter + Markdown	相同
元数据注入	XML 格式	Markdown 列表格式
自动加载	always: true 标记	无此机制
按需指令	一句话: "read its SKILL.md file using the read_file tool"	大段 Progressive Disclosure 说明
子代理 skill	自动继承 XML 摘要	通过 middleware 继承

Nanobot架构: ToolRegistry

ToolRegistry -- 工具注册

管理所有可用工具，提供 JSON Schema 格式的工具定义给 LLM:

```
# 内置工具（自动注册）
ReadFileTool, WriteFileTool, EditFileTool, ListDirTool # 文件操作
ExecTool # Shell 执行
WebSearchTool, WebFetchTool # 联网搜索
SpawnTool # 子 Agent
MessageTool # 消息发送

# 自定义工具注册
loop.tools.register(MyCustomTool())
```

Nanobot架构: ToolRegistry

Thinking: nanobot 内置工具在哪里?

全部在 nanobot/agent/tools/ 目录下:

```
nanobot/agent/tools/  
├─ base.py      # Tool 抽象基类  
├─ registry.py  # ToolRegistry (注册/查找/执行)  
├─ filesystem.py # ReadFileTool, WriteFileTool, EditFileTool, ListDirTool  
├─ shell.py     # ExecTool  
├─ web.py       # WebSearchTool, WebFetchTool  
├─ spawn.py     # SpawnTool  
├─ message.py   # MessageTool  
├─ cron.py      # CronTool (定时任务)  
└─ mcp.py       # MCP 协议工具桥接
```

每个工具都继承自 Tool 基类

```
class Tool(ABC):  
    name: str      # 工具名 (模型调用时用)  
    description: str # 描述 (模型理解用途)  
    parameters: dict # JSON Schema 参数定义  
  
    async def execute(**kwargs) -> Any # 执行逻辑
```

Nanobot架构: ToolRegistry

Thinking: WebSearchTool 怎么实现的?

支持 5 种搜索引擎, 通过 config.json 的 provider 字段切换:

```
async def execute(self, query: str, count: int | None = None, **kwargs: Any) -> str:
    provider = self.config.provider.strip().lower() or "brave"

    if provider == "duckduckgo":
        return await self._search_duckduckgo(query, n)
    elif provider == "tavily":
        return await self._search_tavily(query, n)
    elif provider == "searxng":
        return await self._search_searxng(query, n)
    elif provider == "jina":
        return await self._search_jina(query, n)
    elif provider == "brave":
        return await self._search_brave(query, n)
```

常用搜索引擎API

Provider	API Key	特点
Brave	BRAVE_API_KEY	商业搜索API， Brave 浏览器的搜索引擎， 隐私友好， 搜索结果质量高
Tavily	TAVILY_API_KEY	专为 LLM/Agent 设计， 返回已清洗的 Markdown 格式， 省去额外解析
DuckDuckGo	不需要	自动抓取多个来源， 适合当免费兜底方案
SearXNG	不需要(需自建)	可自行部署的开源项目， 聚合 Google/Bing 等结果， 无单点依赖， 极客/企业自托管首选
Jina	JINA_API_KEY	Jina AI 出品， 除搜索外还支持网页内容摘要、向量化等， 适合 RAG 流程

Nanobot架构: AgentHook

AgentHook -- 生命周期钩子

类似 DeepAgents 的中间件，但更轻量。可在每次迭代的关键节点插入自定义逻辑:

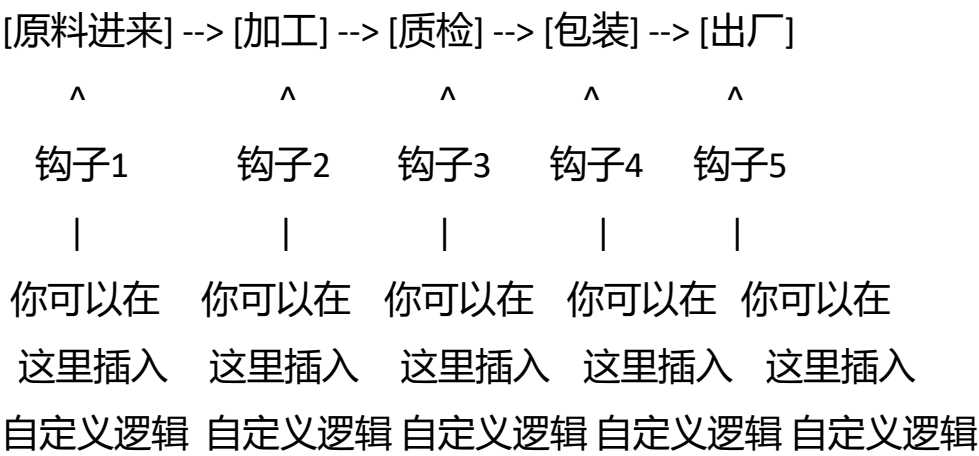
```
class AgentHook:
    async def before_iteration(self, ctx)    # 每轮 LLM 调用前
    async def on_stream(self, ctx, delta)    # 流式输出每个 token
    async def before_execute_tools(self, ctx) # 工具执行前 (可拦截/修改)
    async def after_iteration(self, ctx)     # 每轮结束后
    def finalize_content(self, ctx, content) # 最终输出后处理
```

Nanobot架构: AgentHook

Thinking: 什么是钩子 Hook?

编程中一种经典的设计模式，在一个固定的流程中，预留出一些挂钩点，让外部代码可以在这些点上挂上自己的逻辑。

想象一条流水线（Agent 循环）：



流水线本身不需要改，你只需要把自己的代码挂到对应的钩子上。

Nanobot架构: AgentHook

runner.py 中 Agent 每一轮迭代的完整流程:

```
for iteration in range(max_iterations):
```

```
|
|
|  └─ (1) hook.before_iteration(ctx)    # 钩子: LLM 调用前
|
|
|  └─ (2) 调用 LLM 模型 ───────────
|      |
|      |
|      └─ hook.on_stream(ctx, delta)    # 钩子: 每个 token 流出时
|      └─ hook.on_stream_end(ctx)      # 钩子: 流式结束
|
|
|  └─ if 模型返回了 tool_calls:
|      |
|      |
|      └─ (3) hook.before_execute_tools(ctx) # 钩子: 工具执行前
|
|
|  └─ (4) 执行工具 (read_file/web_search/exec...)
|
|
|  └─ (5) hook.after_iteration(ctx)    # 钩子: 本轮结束
```

```
|
|
|  └─ continue (下一轮迭代)
|
|
|  └─ if 模型返回了最终文本:
|      |
|      |
|      └─ (6) hook.finalize_content(ctx, text) # 钩子: 后处理最终输出
|
|
|  └─ (7) hook.after_iteration(ctx)    # 钩子: 本轮结束
|
|
|  └─ break (结束循环)
```

Nanobot架构：AgentHook

对应源码中的调用位置（精简版）：

```
# runner.py 第 93-249 行的核心循环
for iteration in range(spec.max_iterations):

    await hook.before_iteration(context)      # (1)

    response = await self._request_model(...) # (2) 调 LLM

    if response.has_tool_calls:
        await hook.before_execute_tools(context) # (3)
        results = await self._execute_tools(...) # (4)
        await hook.after_iteration(context)      # (5)
        continue

    clean = hook.finalize_content(ctx, response.content) # (6)
    await hook.after_iteration(context)          # (7)
    break
```


Nanobot架构: AgentHook

基类默认实现：什么都不做

```
class AgentHook:
    """Minimal lifecycle surface for shared runner customization."""
    def wants_streaming(self) -> bool:
        return False

    async def before_iteration(self, context: AgentHookContext) -> None:
        pass

    async def on_stream(self, context: AgentHookContext, delta: str) -> None:
        pass

    async def before_execute_tools(self, context: AgentHookContext) -> None:
        pass

    async def after_iteration(self, context: AgentHookContext) -> None:
        pass

    def finalize_content(self, context: AgentHookContext, content: str | None) -> str | None:
        return content
```

每个方法都是 pass 或直接 return

=> 不挂钩，就什么都不做

钩子的精髓：非侵入式、可选的扩展点。

Skills 渐进式加载机制

SKILL.md 文件格式

```
---  
name: web-search  
description: "联网搜索实时市场信息，获取最新股价、行业动态等"  
keywords: 联网, 搜索, 最新, 实时, 行情  
---
```

web-search 技能指南

适用场景

- 查询某只股票的最新行情
- 获取最新的行业政策

...

Skills 渐进式加载机制

加载流程



Thinking: 为什么渐进式加载重要?

投研Agent 有 8 个技能, 每个 SKILL.md 平均 2000+ 字符。如果全部塞入 prompt, 仅技能指导就要占用 16000+ 字符 (约 4000 tokens) 。

=> 渐进式加载下, prompt 中只放约 500 字符的 XML 摘要, Agent 根据用户问题自主决定读取哪些技能。

=> 运行Agent 时, 日志显示 Agent 主动读取了 investment-research SKILL.md (通过 read_file) , 而不是一开始就加载全部 8 个技能

自定义工具开发

自定义工具开发

- Tool 基类

所有自定义工具继承 `nanobot.agent.tools.base.Tool`, 实现 4 个属性 + 1 个方法

```
from nanobot.agent.tools.base import Tool

class QueryDBTool(Tool):
    @property
    def name(self) -> str:
        return "query_db"

    @property
    def description(self) -> str:
        return "Execute a read-only SQL query against the database."

    @property
    def parameters(self) -> dict:
        return {
```

```
        "type": "object",
        "properties": {
            "sql": {"type": "string", "description": "SQL query"}
        },
        "required": ["sql"]
    }

    @property
    def read_only(self) -> bool:
        return True # 标记为只读, 可安全并行执行

    async def execute(self, **kwargs) -> str:
        sql = kwargs["sql"]
        # 执行查询并返回结果
        ...
```

自定义工具开发

- 工具注册方式

```
# 方式一: 通过 AgentLoop 直接注册
```

```
loop = AgentLoop(bus, provider, workspace, ...)
```

```
loop.tools.register(QueryDBTool(db_path))
```

```
# 方式二: 通过 Nanobot SDK 注册
```

```
bot = Nanobot.from_config("config.json")
```

```
bot._loop.tools.register(ThinkTool())
```

DeepAgents 用 @tool 装饰器定义工具函数，由中间件自动管理;

Nanobot 用类继承方式定义，需要显式注册。类方式更清晰，支持参数验证、只读标记等元数据。

自定义工具开发

- 三个自定义工具实例

工具名	所属示例	功能	特殊属性
query_db	text-to-sql	在 Chinook 数据库上执行 SQL	read_only=True
think_tool	deep-research	反思工具，记录研究进展评估	read_only=True
(无自定义)	content-builder	直接使用内置 web_search + write_file	-

LLM 提供商配置

config.json 配置

```
{
  "agents": {
    "defaults": {
      "model": "qwen-plus",      // 不带 dashscope/ 前缀
      "temperature": 0.1,
      "max_tokens": 8192,
      "timezone": "Asia/Shanghai"
    }
  },
  "providers": {
    "dashscope": {
      "api_key": ""             // 运行时从环境变量注入
    }
  },
}
```

```
  "tools": {
    "web": {
      "search": { "provider": "tavily" }
    },
    "exec": { "enable": true }
  }
}
```


LLM 提供商配置

提供商自动匹配

Nanobot 通过 ProviderSpec 注册表实现模型名到提供商的自动匹配:

```
# providers/registry.py 中的 DashScope 规格
ProviderSpec(
    name="dashscope",
    keywords=("qwen", "dashscope"),    # 模型名含 "qwen" 即匹配
    env_key="DASHSCOPE_API_KEY",
    default_api_base="https://dashscope.aliyuncs.com/compatible-mode/v1",
)
```

模型名用 qwen-plus 而不是 dashscope/qwen-plus。因为 DashScope 的 ProviderSpec 没有设置 strip_model_prefix=True，带前缀的模型名会原样发送到 API 导致 "model not found" 错误。

LLM 提供商配置

运行时 API Key 注入

```
# agent.py 中从环境变量注入（不硬编码到 config.json）
config = load_config("config.json")
config.providers.dashscope.api_key = os.environ["DASHSCOPE_API_KEY"]
config.tools.web.search.api_key = os.environ.get("TAVILY_API_KEY", "")

provider = _make_provider(config) # 自动选择 OpenAICompatProvider
```

打卡：舆情感知与事件驱动



搭建AI投研中的sentiment-analysis 技能，完成以下任务：

- 运行恐慌指数监控，查看当前全球市场情绪（VIX + 10年期国债）
- 运行 Polymarket 监控，搜索一个你关心的地缘政治话题
- 选择一只你关注的股票，获取最近7天的新闻并运行情感分析
- 运行事件检测，查看是否有重大事件
- 如果检测到事件，尝试联动 read-pdf 和 write-report 做深度分析



Thank You
Using data to solve problems

